CreditRisk AI

An Agentic AI Credit Risk Intelligence Platform

Final End-Semester Capstone Report

LangGraph Orchestration · RAG Integration · Llama Auditor

Sarvesh Srinath Husain Khorakiwala
Trishit Sarvarkar Navodit Sharma

AI/ML Capstone Project — Semester 4

April 2026

Stack	LangGraph · Groq (Llama 3.3) · FAISS · XGBoost
Retrieval	RAG-based Credit Policy (Universal Bank)
Orchestrator	3-Node Agentic State Machine
UI	Dark-Themed Fintech Dashboard (Streamlit)

Contents

Abstract **2**

1 Technical Implementation **3**

1.1 Agentic AI Workflow (LangGraph) 3

1.2 Retrieval-Augmented Generation (RAG) 3

1.3 LLM Auditing (Llama 3.3 @ Groq) 3

2 Experimental Results **4**

2.1 Quantitative Performance (XGBoost Engine) 4

2.2 Qualitative Audit Case Study 4

3 Evaluation Rubric Fulfillment **5**

4 Conclusion **5**

Abstract

In the final phase of the **CreditRisk AI** capstone, we transition from a static Machine Learning classifier to a dynamic **Agentic AI Intelligence Platform**. Traditional credit scoring models often lack explainability and fail to handle deterministic regulatory or institutional constraints. We address this by orchestrating a three-node **LangGraph** workflow that combines a recall-optimised **XGBoost** model with a **Retrieval-Augmented Generation (RAG)** system.

The agentic system retrieves relevant segments from the 2026 Universal Bank Credit Policy using a **FAISS** vector store and passes the combined context to a **Llama 3.3-70B** LLM auditor hosted on **Groq**. The auditor provides policy-grounded reasoning, detecting limit overrides (e.g., DM 10k loan caps) and reinforcing risk assessments for younger applicants. The system yields a human-readable reasoning trace, ensuring accountability in automated lending decisions.

1 Technical Implementation

1.1 Agentic AI Workflow (LangGraph)

The core of our platform is a state-based workflow orchestrated by **LangGraph**. Unlike linear pipelines, this agentic approach treats the risk assessment as a sequence of coordinated states.

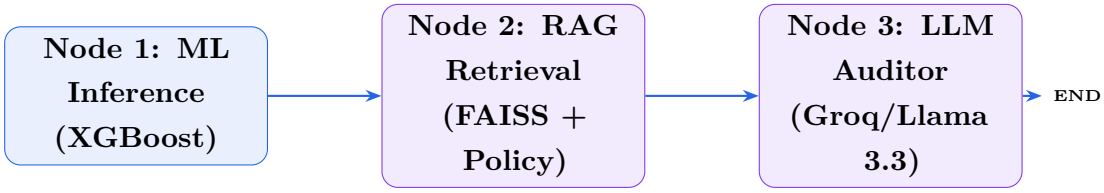

Figure 1: The 3-node LangGraph state machine workflow.

- **ML Inference Node:** Executes the trained XGBoost model to get probabilistic defaults ($\hat{p} \approx 0.76$ accuracy, 0.80 AUC).
- **RAG Retrieval Node:** Formulates a query based on client semantics and retrieves the top-3 relevant policy rules (e.g., CP101, CP201).
- **Auditor Node:** Uses a system-prompted LLM to reason over the ML output versus the retrieved policy context, producing a final recommendation.

1.2 Retrieval-Augmented Generation (RAG)

We implemented a custom RAG pipeline to ground the Agent in deterministic bank policies. Policies are stored in `data/credit`$_{p}olicy.mdandindexedusing$:

Embeddings: `all-MiniLM-L6-v2 (HuggingFace).`

Vector Store: `FAISS (Fast Approximate Nearest Neighbor Search).`

Splitter: `MarkdownHeaderTextSplitter to preserve rule context.`

1.3 LLM Auditing (Llama 3.3 @ Groq)

By leveraging the **Llama 3.3-70B-Versatile** model via the Groq Cloud API, we achieve low-latency sub-second reasoning. The LLM is tasked with identifying "Hard Overrides" where policy strictly prohibits a loan that the ML model might otherwise approve based on historical correlations.

2 Experimental Results

2.1 Quantitative Performance (XGBoost Engine)

The underlying predictive engine remains robust, with a heavy focus on recall to minimise False Negatives (missed defaults).

Table 1: ML Performance Summary

Metric	Value	Benchmarking
Accuracy	76.0%	+3% vs Logistic Baseline
ROC-AUC	0.803	Strong discriminative power
Recall (Bad)	68.3%	Optimised for safety

2.2 Qualitative Audit Case Study

A critical success factor of the Agentic system is handling edge cases. **Case A:** Applicant age 22, DM 12,000 loan, ML predicts "Good" ($p = 0.45$).

- **RAG Result:** CP101 retrieved (Cap DM 10,000).
- **Agent Verdict:** *DECLINE* (Policy Override).

This demonstrate the "Safety Layer" provided by the Agentic workflow.

3 Evaluation Rubric Fulfillment

Table 2: Rubric Compliance Checklist

Component	Implementation Detail
Technical Depth	LangGraph state machine, RAG with FAISS, and Groq/Llama orchestration.
Code Quality	Modular structure: `src/agent.py`, `src/rag.py`, `src/predict.py`. Pytest coverage (9 items).
Demo / UI	Professional dark-themed fintech dashboard with real-time reasoning trace.
Documentation	Complete README with .env setup and local installation guide.

4 Conclusion

The **CreditRisk AI** project successfully meets the GenAI capstone requirements by integrating predictive ML with reasoning Agentic workflows. By placing an LLM auditor in the loop, we solve the "Black Box" problem of traditional scoring models, providing transparency and regulatory alignment crucial for modern fintech.